

# Ghid complet de învățare pentru licență

Platformă web sigură pentru planificarea, monitorizarea și analiza antrenamentelor de forță

**Scop: să poți explica proiectul coerent, de la povestea aplicației până la deciziile tehnice.**

Folosește ghidul ca material de apărare, nu ca text de memorat. Învăță problemele, soluțiile și motivul pentru care soluțiile contează.

Versiune generată pentru pregătirea prezentării din 2 iulie 2026.

## 0. Cum să folosești ghidul în 5 ore

Nu încerca să memorezi tot. Trebuie să poți reconstrui sistemul din cap. Ordinea optimă este: fluxul principal, apoi deciziile tehnice, apoi întrebările probabile.

| Timp   | Ce faci                                                                         | Rezultat                                               |
|--------|---------------------------------------------------------------------------------|--------------------------------------------------------|
| 30 min | Citești harta aplicației și povestea de 2 minute.                               | Poți spune ce face aplicația fără să te blochezi.      |
| 70 min | Înveți fluxul principal: planificare -> sesiune -> istoric -> analytics/search. | Poți explica produsul ca utilizator și ca dezvoltator. |
| 90 min | Înveți pilonii tehnici: snapshot-uri, securitate, OpenSearch, testare.          | Poți răspunde la întrebările grele.                    |
| 60 min | Înveți marketplace, coaching, analizator, anexe/API/config.                     | Acoperi modulele secundare fără goluri.                |
| 50 min | Răspunzi cu voce tare la întrebările din final.                                 | Transformi cunoștințele în apărare orală.              |

**Regula principală: la fiecare secțiune trebuie să știi: problema, soluția implementată și de ce soluția este corectă.**

## 1. Povestea proiectului

### Explicația de 30 de secunde

Am construit o platformă web full-stack pentru planificarea, executarea și analiza antrenamentelor de forță.

Utilizatorul își creează resursele de planificare, pornește sesiuni live, salvează istoricul, urmărește progresul, publică sau copiază programe din marketplace și poate colabora controlat cu un antrenor. Accentul proiectului este pe corectitudinea istoricului, securitate multi-user și căutare scalabilă.

### Explicația de 2 minute

- Aplicația începe cu date editabile: exerciții, rutine, săli, echipamente și template-uri de antrenament.
- Un template este un program pe zile. Fiecare zi conține exerciții planificate, valori țintă și rutine de început/final.
- Când utilizatorul pornește un antrenament, o zi de template devine o sesiune live.
- La pornirea sesiunii, aplicația copiază datele importante în snapshot-uri: nume de exerciții, grupe musculare, rutină, sală, echipament și câmpuri planificate.
- După finalizare, sesiunea intră în istoric și este folosită pentru analytics și căutare.
- Marketplace-ul permite publicarea, votarea, salvarea și copierea template-urilor publice.
- Modul de coaching permite unui coach să vadă datele clientului doar dacă există o relație activă acceptată.
- OpenSearch este folosit ca model de citire pentru căutare full-text, fuzzy, fațete și evidențiere, dar PostgreSQL rămâne sursa de adevăr.

### Harta mentală

```
Resurse de planificare
-> exerciții / rutine / săli / echipamente
-> template-uri pe zile
-> sesiune live pornită dintr-o zi
-> seturi efectuate + note + echipament
-> finalizare / anulare
-> istoric + analytics + căutare
```

Module laterale:

```
marketplace = publish / vote / save / use(deep copy)
coaching = invite / accept / active relationship / read-only access
analyzer = scor determinist al structurii programului
security = JWT + ownership + RBAC/ReBAC + IDOR protection
```

## 2. Capitolul 1 - Introducere

### Ce trebuie să știi

- Lucrarea este aplicativă: o aplicație web full-stack, nu doar un studiu teoretic.
- Domeniul este fitness/strength training, dar tema reală este construirea unui sistem corect, sigur și scalabil.
- Problema principală: datele de planificare se schimbă, dar istoricul trebuie să rămână fidel momentului execuției.
- A doua problemă: aplicația este multi-user, deci autentificarea nu este suficientă; fiecare resursă trebuie verificată.
- A treia problemă: căutarea devine dificilă când apar multe template-uri și sesiuni, deci OpenSearch este folosit ca model derivat.

### Contribuția proprie

- Backend Spring Boot organizat modular pe funcționalități.
- Schemă PostgreSQL gestionată prin Flyway, cu constrângeri, indecși parțiali, soft delete și reguli de concurență.
- Model de sesiuni cu snapshot-uri, ca istoricul să nu depindă de date editabile.
- Autentificare și autorizare cu JWT, refresh-token rotation, BCrypt, ownership checks, RBAC/ReBAC.
- Funcționalități complete: exerciții, rutine, săli, echipamente, template-uri, sesiuni live, istoric, analytics, marketplace, coaching.
- Analizator determinist de programe, fără pretenții medicale.
- OpenSearch pentru căutare full-text peste template-uri și istoric.
- Benchmark OpenSearch vs PostgreSQL până la 2.000.000 de documente.

**Corecție de ținut minte: în PDF-ul exportat apare greșit că secțiunea de structură trimite tot la Capitolul 2. Corect este: Capitolul 2 = preliminarii, Capitolul 3 = implementare/contribuție, Capitolul 4 = concluzii.**

## 3. Capitolul 2 - Preliminarii

### Aplicații web și sursa de adevăr

- Frontend-ul React trimite cereri JSON către un API REST.
- Backend-ul Spring Boot aplică regulile de domeniu și persistă datele.
- PostgreSQL este sursa unică de adevăr: constrângeri, tranzacții, chei externe, date istorice.
- Hibernate nu modifică schema automat; schema este controlată cu Flyway.

### Date de planificare vs date istorice

- Datele de planificare sunt editabile: template-uri, exerciții, rutine, săli, echipamente.
- Datele istorice trebuie să rămână stabile: sesiuni, exerciții din sesiune, rutine din sesiune, seturi.
- De aceea aplicația copiază valori în snapshot-uri la pornirea sesiunii.
- Legăturile către resursele originale pot fi nullable și auxiliare; istoricul afișat vine din snapshot.

Corect: `workout_sets -> session_exercises -> workout_sessions`  
Greșit: `workout_sets -> template_day_exercises`

### Autentificare, autorizare și IDOR

- Autentificarea răspunde la întrebarea: cine este utilizatorul?
- Autorizarea răspunde la întrebarea: are voie utilizatorul să acceseze această resursă?
- IDOR apare când un utilizator modifică un ID în cerere și accesează resursa altcuiva.
- Backend-ul nu se bazează pe userId trimis de frontend; folosește user-ul extras din JWT.

- Pentru resursele altui utilizator se întoarce 404, nu 403, ca existența resursei să nu fie confirmată.

## **RBAC vs ReBAC**

- RBAC = Role-Based Access Control. Exemple: /api/admin/\*\* cere ADMIN, /api/coach/\*\* cere COACH.
- ReBAC = Relationship-Based Access Control. Exemplu: coach-ul vede un client doar dacă relația coach-client este ACTIVE.
- Rolul COACH este necesar, dar nu suficient. Trebuie și relația activă.

## **Căutare full-text și model de citire**

- PostgreSQL rămâne autoritativ.
- OpenSearch este o proiecție denormalizată, reconstrucibilă, optimizată pentru căutare.
- Căutarea folosește text, filtre, fațete, evidențiere și toleranță la greșeli.
- Rezultatele întoarse de OpenSearch sunt revalidate în PostgreSQL.

## **Analizator determinist**

- Analizatorul nu este AI și nu este sfat medical.
- El aplică reguli explicabile asupra structurii unui template.
- Exemple: volum, frecvență, acoperire musculară, push/pull, lower/upper, pauze și lungimea sesiunilor.

## 4. Capitolul 3 - Implementarea aplicației

### 3.1 Vedere de ansamblu

- Platforma urmărește ciclul complet al unui antrenament: planificare, execuție, istoric, analiză și colaborare.
- Backend-ul este organizat pe pachete de funcționalitate: auth, exercise, routine, gym, template, session, history, analytics, marketplace, analyzer, search, coaching, shared.
- În fiecare zonă există separare între domeniu, servicii, repository-uri și controllere.
- Frontend-ul React este și el împărțit pe funcționalități și consumă DTO-uri, nu entități JPA.

### 3.2 Flux complet de utilizare

- Utilizatorul se autentifică.
- Își creează exerciții, rutine, săli și echipamente.
- Construiește template-uri pe mai multe zile.
- Pornește o sesiune live dintr-o zi de template.
- În sesiune adaugă seturi, greutate, repetări, durată, distanță, RPE, note și echipament.
- Poate adăuga exerciții extra neplanificate.
- Finalizează sau anulează sesiunea.
- Sesiunea finalizată intră în istoric, analytics și căutare.
- Template-urile pot fi publicate în marketplace.
- Coach-ul poate vedea datele clientului doar cu relație activă.

### 3.3 Cerințe funcționale

| Zonă              | Ce trebuie să facă aplicația                                                                       |
|-------------------|----------------------------------------------------------------------------------------------------|
| Auth              | Register, login, refresh, logout, /me; identificare prin JWT.                                      |
| Planificare       | Exerciții oficiale/custom, rutine, săli, echipamente.                                              |
| Template-uri      | Creare/editare zile, exerciții planificate, rutine, agregate pentru căutare.                       |
| Sesiuni           | Pornire, un singur antrenament activ, seturi, extra exercises, finish/cancel, note.                |
| Istoric/analytics | Listă istoric, detalii sesiune, volum, frecvență, distribuție, best sets, Epley 1RM.               |
| Marketplace       | Publicare, listare, sortare, vot, salvare, deep copy.                                              |
| Analyzer          | Scor 0-100, sub-scoruri, avertismente, puncte forte, limitări.                                     |
| Coaching          | Invitație, accept/reject, revoke, acces read-only cu relație activă.                               |
| Căutare           | Template-uri personale/publice, istoric propriu, istoric client, filtre, fațete, highlight, fuzzy. |

### 3.3 Cerințe nefuncționale

- Corectitudine: PostgreSQL + Flyway + constrângeri + snapshot-uri.
- Securitate: JWT, refresh cookie, hashing, ownership, RBAC/ReBAC.
- Izolare multi-user: fiecare utilizator vede/modifică doar resursele sale; public doar controlat.
- Scalabilitate: OpenSearch pentru interogări text/fuzzy la volum mare.
- Consistență: OpenSearch derivat, PostgreSQL autoritativ, indexing after commit.
- Testabilitate: Testcontainers cu PostgreSQL real și OpenSearch real, fără H2.
- Menținabilitate: cod modular, DTO-uri, repository-uri clare, Docker Compose.

## 5. Modelul de date, snapshot-uri și securitate

### 3.4 Modelul de date și istoricul prin snapshot-uri

- V1 definește schema inițială cu 28 de tabele.
- V2 înserează exercițiile oficiale și mapările lor către grupele musculare. Atenție: grupele musculare sunt create/seeded în V1.
- V3 adaugă indexul unic parțial pentru un singur antrenament activ per utilizator.
- Zona de planificare: `workout_templates`, `template_days`, `template_day_exercises`, `routines`, `gyms`, `equipment`.
- Zona istorică: `workout_sessions`, `session_exercises`, `session_routines`, `workout_sets`.

### Ce se copiază în snapshot-uri

- La sesiune: numele template-ului, numele zilei, sala, exercițiile, tipul exercițiului, valorile planificate, rutina, grupele musculare.
- La set: echipamentul poate fi snapshot-uit prin `equipment_name_snapshot`.
- Cheile către template/exercițiu/rutină/sală/echipament pot rămâne nullable, dar afișarea istoricului nu depinde de ele.

### De ce contează snapshot-urile

- Dacă utilizatorul redenumeste Bench Press, o sesiune veche trebuie să afișeze numele de atunci, nu numele nou.
- Dacă un template este editat, istoricul nu trebuie rescris retroactiv.
- Dacă o sală sau un echipament este șters logic, istoricul rămâne inteligibil.
- Căutarea peste istoric folosește snapshot-uri, deci păstrează adevărul istoric.

### 3.5 Constrângeri, ștergere logică și concurență

- Soft delete pe resurse editabile: `exercises`, `routines`, `gyms`, `equipment`, `templates`.
- Constrângeri CHECK pentru enumerări, RPE, durate, `published_at`, contoare nenegative.
- Index unic parțial: un singur workout IN\_PROGRESS pe user.
- Indecși unici parțiali pentru nume active, ca soft delete să nu blocheze re folosirea numelui.
- UNIQUE(`session_exercise_id`, `set_number`) apără împotriva dublării seturilor la concurență.

### 3.6 Securitate și controlul accesului

- Access token JWT semnat HS256, durată scurtă.
- Refresh token opac în cookie HttpOnly, SameSite=Strict, path=/api/auth.
- În DB se stochează doar SHA-256 hash pentru refresh token.
- La refresh, token-ul vechi este revocat și se emite o pereche nouă.
- Parolele sunt hash-uite cu BCrypt.
- Security config este stateless, fără sesiuni HTTP server-side.
- Rute publice: `auth/register/login/refresh/logout` și `/api/health`.
- `/api/admin/**` cere ADMIN; `/api/coach/**` cere COACH; restul `/api/**` cere autentificare.
- Frontend-ul nu trimite `userId` pentru operații personale; backend-ul ia utilizatorul din JWT.
- Ownership checks se fac în servicii/repository-uri; resursele altui user întorc 404.

### 404 vs 403

- 403 ar confirma că resursa există, dar utilizatorul nu are acces.

- 404 ascunde existența resursei și reduce riscul de enumerare IDOR.
- Excepție: non-coach pe /api/coach/\*\* primește 403 prin RBAC, pentru că nu este o resursă personală specifică.

### **RBAC/ReBAC în coaching**

- RBAC: utilizatorul trebuie să aibă rolul COACH pentru rutele /api/coach/\*\*.
- ReBAC: pentru datele unui client trebuie să existe relație ACTIVE între coach și client.
- Fără relație activă, clientul este tratat ca inaccesibil și se întoarce 404.

## 6. Funcționalități principale

### 3.7 Planificare și execuție

- Exerciții: oficiale vizibile tuturor și custom vizibile doar proprietarului.
- Nu spune prea absolut că fiecare exercițiu custom are obligatoriu grupe musculare; backend-ul permite listă goală.
- Rutine: texte reutilizabile de început/final.
- Săli și echipamente: personale, echipamentele aparțin unei săli.
- Template-uri: programe pe zile, zile cu exerciții ordonate, valori planificate și rutine.
- După modificarea exercițiilor din template se recalculează agregatele: muscle groups, official exercise ids, exercise names.
- Sesiune live: pornită dintr-o zi de template, creează snapshot-uri, permite seturi și extra exercises.
- După finish/cancel, sesiunea devine terminală. Seturile nu se mai modifică, dar notele sesiunii pot fi editate.

### 3.8 Istoric și analytics

- Istoricul listează sesiunile utilizatorului descrescător după data de început.
- Pentru sumarul unei pagini se evită N+1 printr-o interogare batch pentru exerciseCount, setCount, totalVolume.
- Analytics folosește sesiunile FINISHED.
- Indicatori: volum pe zi, antrenamente pe săptămână, distribuția seriilor pe grupa musculară primară, cele mai bune seturi, estimare 1RM cu formula Epley.

## Marketplace

- Un template privat poate fi publicat doar dacă nu este gol.
- Marketplace-ul listează template-uri publice și permite sortare după newest, top/scor, trending.
- Utilizatorii pot vota, salva și copia template-uri publice.
- Self-vote este respins.
- Save este idempotent și contoarele nu devin negative.
- Deep copy creează un template privat nou, cu zile, exerciții, rutine și snapshot-uri proprii.
- Referințele la exerciții oficiale pot fi păstrate; referințele custom/străine/deleted sunt eliminate, dar snapshot-urile rămân.
- template\_stats ține contoare, iar template\_use\_events ține evenimente de folosire/copiere.

## 7. Analizator, coaching și căutare

### 3.9 Analizatorul de programe

- Endpoint: GET /api/templates/{id}/analysis.
- Citește structura printr-un model de aplicație, nu direct din DTO-uri web.
- Produce scor 0-100, categorie, sub-scoruri, warnings, strengths, limitations.
- Sub-scoruri: volum și acoperire, frecvență, echilibru, designul sesiunii, specificitate/pauză.
- Volumul ia în calcul doar exerciții de forță.
- Pondere musculară: PRIMARY = 1.0, SECONDARY = 0.5.
- Exercițiile fără seturi planificate sunt marcate ca date incomplete, nu li se inventează volum.
- Categori: WELL\_STRUCTURED >= 80, DECENT\_STRUCTURE >= 55, NEEDS\_REVIEW sub 55.



- Limitări explicite: nu vede tehnică, recuperare, somn, alimentație, efort real; nu oferă sfat medical.

### 3.10 Modul de coaching

- Coach-ul trimite invitație către client.
- Clientul acceptă sau respinge.
- Relația poate fi PENDING, ACTIVE, REJECTED, REVOKED.
- Un index unic parțial permite o singură relație vie pentru aceeași pereche.
- DB interzice ca un utilizator să fie propriul coach.
- Accesul coach-ului este read-only: lista de clienți, istoric, analytics, detalii sesiune, search în istoricul clientului.
- Nu există endpoint prin care coach-ul modifică sesiunile sau template-urile clientului.

### 3.11 OpenSearch ca model de citire

- OpenSearch nu este sursă de adevăr; PostgreSQL este autoritativ.
- Există două indexuri/alias-uri logice: templates și workout\_sessions.
- Documentele sunt denormalizate: nume, descrieri, exerciții, grupe musculare, săli, echipamente, note, scor analiză, popularitate.
- Interogarea folosește multi\_match cu ponderi: numele contează mai mult decât descrierea; în istoric exercițiul contează mai mult decât nota.
- Fuzziness=AUTO este folosit pentru query-uri de cel puțin 3 caractere.
- Sinonime: pecs -> chest, ohp -> overhead press, rdl -> romanian deadlift etc.
- Rezultatele pot include fațete și highlight cu <mark>.
- Indexarea se face după commit, prin evenimente Spring, ca rollback-ul să nu ajungă în index.
- Structural update reconstruiește documentul; stats update modifică doar contoarele de popularitate.
- Dacă indexing-ul eșuează, eroarea se loghează, dar PostgreSQL rămâne corect.
- Security layer 1: filtru obligatoriu în OpenSearch: PUBLIC pentru marketplace, ownerId pentru resurse personale/istoric.
- Security layer 2: revalidare în PostgreSQL înainte de răspuns.
- Reindex-ul manual creează index nou, îl umple din PostgreSQL și schimbă alias-ul atomic. Endpoint-ul cere ADMIN.
- Dacă search e dezactivat sau OpenSearch cade, /api/search răspunde 503, iar UI-ul poate reveni la liste SQL.

## 8. Benchmark, testare și concluzii

### 3.12 Evaluarea OpenSearch vs PostgreSQL

- Benchmark-ul compară OpenSearch cu PostgreSQL bine indexat, nu cu o variantă slabă.
- PostgreSQL folosește tsvector ponderat, GIN, ts\_rank\_cd, pg\_trgm și GROUP BY pentru fațete.
- OpenSearch folosește mapările și interogările aplicației.
- Corpusul este generat determinist de la 1.000 la 2.000.000 de documente.
- S-au măsurat: două full-text, fuzzy typo, filtrare, fațete, timp de construcție, dimensiune pe disc.

| Scenariu la 2M                     | PostgreSQL p50 | OpenSearch p50 |
|------------------------------------|----------------|----------------|
| Full-text bench press              | 1864 ms        | 64 ms          |
| Full-text squat                    | 1437 ms        | 4 ms           |
| Fuzzy bench                        | 15825 ms       | 4 ms           |
| Filtrat press + INTERMEDIATE + PPL | 2295 ms        | 7 ms           |
| Fațetă dificultate                 | 1193 ms        | 2 ms           |
| Build index                        | 239 s          | 115 s          |
| Dimensiune disc                    | 2632 MB        | 739 MB         |

**Interpretare: PostgreSQL este mai simplu și uneori mai rapid la colecții mici. OpenSearch devine justificat la scară și mai ales pentru fuzzy search.**

### 3.13 Testare

- Suita are 128 de teste automate.
- Testele de integrare folosesc PostgreSQL 16 prin Testcontainers.
- Testele de search pornesc și un container OpenSearch real.
- Nu se folosește H2 și nu se folosesc fake repositories.
- Sunt testate: auth, exerciții, rutine, săli/equipment, template-uri, sesiuni, istoric/analytics, marketplace, coaching, analyzer, search.
- Sunt verificate reguli critice: IDOR, o singură sesiune activă, snapshot stability, sesiune finalizată fără modificări de seturi, coach active relationship, marketplace copy, search isolation.

## Capitolul 4 - Concluzii

- Rezultatul este o platformă completă, nu un CRUD.
- Cea mai importantă idee tehnică: datele editabile sunt separate de istoricul executat.
- Securitatea combină token-uri, roluri, ownership și relații active.
- OpenSearch este integrat ca model derivat, nu ca sursă de adevăr.
- Benchmark-ul arată când un motor separat merită costul operațional.
- Direcții viitoare: notificări, comentarii coach-client, assignări de template-uri, outbox robust, grafice mai avansate, eventual cache Redis - dar Redis nu este implementat în teză.

## 9. Anexe și API

### Anexa 1 - Schema bazei de date

- 28 de tabele în V1.

- Zone: auth/coaching, catalog/planificare, programe, sesiuni istorice, marketplace, extensii pregătite.
- Extensii pregătite în schemă: coach\_session\_comments, coach\_template\_assignments, template\_analysis\_results, outbox\_events.
- Atenție: dacă ești întrebat, unele extensii sunt pregătite la nivel de schemă, nu neapărat funcționalități complete expuse în UI.

## Anexa 2 - Cod reprezentativ

- Analizatorul: reguli deterministe pentru scor și warning-uri.
- Security config: endpoint-uri publice, admin, coach, autentificare pentru restul.
- Migrații SQL: constrângeri, indecși parțiali, CHECK-uri.

## Anexa 3 - Benchmark complet

Tabelele complete susțin concluzia din corpul lucrării: OpenSearch nu e introdus ca ornament, ci pentru un caz măsurat unde câștigă clar la scară, mai ales fuzzy search.

## Anexa 4 - API corect de știut

```
Auth:
POST /api/auth/register, /login, /refresh, /logout
GET /api/auth/me

Planning:
GET /api/exercises, GET/POST /api/routines
GET/POST/PUT/DELETE gyms/equipment

Templates:
GET/POST /api/templates
GET/PUT/DELETE /api/templates/{id}
POST /api/templates/{id}/publish|unpublish

Sessions:
POST /api/workouts/start
GET /api/workouts/active, GET /api/workouts/{id}
POST /api/workouts/{id}/finish|cancel
POST /api/session-exercises/{id}/sets
PUT/DELETE /api/sets/{id}

Marketplace/Search/Coaching:
GET /api/marketplace/templates
POST /api/marketplace/templates/{id}/vote|save|use
GET /api/templates/{id}/analysis
GET /api/search/templates, /api/search/workouts
POST /api/admin/search/reindex
GET /api/coach/clients/{clientId}/history
GET /api/coach/clients/{clientId}/analytics
GET /api/coach/clients/{clientId}/sessions/{sessionId}
GET /api/coach/clients/{clientId}/search/workouts
```

## Anexa 5 - Configurație și rulare

- docker-compose pornește PostgreSQL 16, OpenSearch 2.13.0 și aplicația Spring Boot.
- Frontend-ul Vite este construit în build-ul Maven și inclus în jar.
- Aplicația servește UI + API la http://localhost:8080.
- Frontend dev server poate rula separat pe 5173 și proxy-uieste /api către 8080.
- Hibernate ddl-auto=validate; Flyway gestionează schema.
- app.search.enabled este false implicit, true în compose prin APP\_SEARCH\_ENABLED=true.

## 10. Întrebări probabile și răspunsuri scurte

### Q: Care este ideea principală a lucrării?

O platformă full-stack pentru planificarea, executarea și analiza antrenamentelor, construită cu accent pe corectitudinea istoricului, securitate multi-user și căutare scalabilă.

### Q: De ce ai folosit snapshot-uri?

Pentru că datele de planificare sunt editabile, iar istoricul trebuie să rămână fidel momentului execuției. Fără snapshot-uri, o sesiune veche s-ar schimba când modific template-ul/exercițiul/sala.

### Q: De ce seturile nu sunt legate direct de template\_day\_exercises?

Pentru că template-ul este planul editabil, nu execuția istorică. Seturile aparțin unei session\_exercise, care aparține unei workout\_session.

### Q: De ce PostgreSQL este sursa de adevăr?

Pentru integritate: tranzații, chei externe, constrângeri, indecși și date istorice corecte. OpenSearch este doar o proiecție derivată.

### Q: De ce OpenSearch?

Pentru că oferă căutare full-text, fuzzy, fațete, highlight și performanță mai bună la scară. Benchmark-ul arată clar câștigul la corpus mare.

### Q: Ce se întâmplă dacă OpenSearch cade?

Datele rămân corecte în PostgreSQL. Endpoint-urile de search pot întoarce 503, iar listele SQL rămân disponibile. Indexul poate fi reconstruit.

### Q: Cum previi IDOR?

Backend-ul ia user-ul din JWT, nu din body. Resursele sunt încărcate owner-scoped, iar resursele altui utilizator întorc 404.

### Q: Ce este RBAC vs ReBAC?

RBAC verifică rolul, de exemplu COACH sau ADMIN. ReBAC verifică relația, de exemplu coach-ul are acces doar la clientul cu relație ACTIVE.

### Q: Este analizatorul AI?

Nu. Este determinist, rule-based, explicabil. Nu oferă sfat medical și nu înlocuiește un antrenor.

### Q: Care este cea mai importantă contribuție?

Integrarea funcționalităților într-un sistem coerent: istoric stabil prin snapshot-uri, securitate multi-user, marketplace, coaching, analyzer și search derivat cu revalidare în PostgreSQL.

### Q: De ce Testcontainers și nu H2?

Pentru că regulile importante depind de PostgreSQL real: indecși parțiali, CHECK-uri, ON DELETE SET NULL, query-uri native, pg\_trgm, tranzații.

### Q: Ce ai face mai departe?

Notificări, comentarii coach-client, assignări de template-uri, outbox mai robust, grafice mai avansate, eventual cache Redis pentru ranking-uri sau rezultate frecvente.

## 11. Ce să nu spui greșit

- Nu spune că Redis este implementat. Nu este în thesis PDF și nu este implementat.
- Nu spune că OpenSearch este sursa de adevăr. Sursa de adevăr este PostgreSQL.
- Nu spune că analizatorul este AI sau că oferă recomandări medicale.
- Nu spune că V2 creează grupele musculare. V1 le creează și le inserează; V2 inserează exercițiile oficiale și mapările.
- Nu spune că orice coach poate vedea orice utilizator. Coach-ul vede doar clienții cu relație ACTIVE.
- Nu spune că frontend-ul trimite userId pentru ownership. Backend-ul determină user-ul din JWT.
- Nu spune că un template copiat rămâne dependent de original. Copia devine privată și independentă.
- Nu spune absolut că orice exercițiu custom are grupe musculare obligatorii; oficialele au, custom poate avea listă goală.

### Fraze sigure

- PostgreSQL rămâne autoritativ, iar OpenSearch este un model de citire derivat.
- Snapshot-urile păstrează adevărul istoric al sesiunii.
- Rolul COACH nu este suficient; accesul la client cere relație activă.
- Rezultatele OpenSearch sunt revalidate în PostgreSQL pentru securitate și consistență.
- Analyzer-ul este determinist și explicabil, nu medical și nu AI.

## 12. Checklist pentru mâine

### Înainte de prezentare

- Pot explica fluxul complet fără să mă uit: planning -> template -> session -> history -> analytics/search.
- Pot explica snapshot-urile cu un exemplu: redenumesc Bench Press după o sesiune veche.
- Pot explica IDOR cu un exemplu: schimb ID-ul din URL și backend-ul întoarce 404.
- Pot explica RBAC/ReBAC în 20 secunde.
- Pot explica de ce OpenSearch există și de ce nu înlocuiește PostgreSQL.
- Pot spune că testele sunt cu PostgreSQL/OpenSearch reale prin Testcontainers.
- Știu cele patru mici corecții: chapter refs, V2 wording, coach routes, custom muscle groups.

### Deschidere de prezentare

Bună ziua. Lucrarea mea prezintă o platformă web pentru planificarea, monitorizarea și analiza antrenamentelor de forță. Nu am tratat aplicația ca pe un simplu CRUD, ci ca pe un sistem multi-user în care istoricul trebuie să rămână corect după editări, accesul la date trebuie controlat, iar căutarea trebuie să rămână utilizabilă pe măsură ce volumul de date crește.

### Închidere de prezentare

În concluzie, contribuția principală este integrarea mai multor probleme reale într-o platformă coerentă: snapshot-uri pentru istoric stabil, securitate bazată pe JWT, ownership și relații coach-client, un marketplace cu copii independente, un analizator determinist și un model de citire OpenSearch revalidat în PostgreSQL. Rezultatul este o aplicație funcțională și extensibilă, cu reguli verificate prin teste de integrare pe infrastructură reală.